Science-Bitch: A Comprehensive Research Guide

Summary: Science-Bitch is a comprehensive command-line tool that implements the complete scientific method workflow for systematic hypothesis testing, controlled experimentation, and...

Generated: 2026-01-12 23:53 UTC | **Ideas Extracted:** 68

What Happened

> Science-Bitch is a comprehensive command-line tool that implements the complete scientific method workflow for systematic hypothesis testing, controlled experimentation, and evidence-based conclusions. This research booklet provides a complete guide to understanding, using, and extending the Science-Bitch system.

Science-Bitch is a command-line tool that provides a structured, systematic approach to scientific research and experimentation. It implements the complete scientific method workflow, from initial hypothesis formation through experiment execution, data collection, analysis, and report generation.

> The scientific method is a systematic approach to understanding the natural world through observation, hypothesis formation, experimentation, and analysis. Science-Bitch implements this complete cycle:.

Complete Scientific Method Implementation: Full workflow from hypothesis formation to conclusion State Capture System: Automatic tracking of system states before (A) and after (B) experiments Systematic Data Collection: Comprehensive data collection during experiments (C) Hypothesis Testing Framework: Structured approach to forming, testing, and verifying hypotheses Automated Report Generation: Professional PDF reports with full documentation Evidence-Based Analysis: Systematic analysis with traceable evidence chains.

Researchers: Scientists conducting systematic experiments Developers: Engineers testing hypotheses about code/system behavior Data Scientists: Analysts needing structured experimentation workflows Students: Learners practicing the scientific method Teams: Groups needing standardized research processes.

> Core functionality from `scientificmethodtool/` : Hypothesis formation and validation Experiment design Data collection frameworks Analysis tools.

Process: Define experiment structure Specify variables and their types Design data collection methods Plan state capture points (A and B) Define success criteria.

> Complete experiment documentation: Hypothesis and design State snapshots (A and B) Collected data (C) Analysis results Conclusions.

Core functionality from `scientificmethodtool/` Hypothesis framework Experiment design Analysis tools.

Be Specific: Clear, unambiguous statements Make Testable: Can be verified or refuted Define Variables: Clear independent/dependent variables Set Criteria: What constitutes verification?

Systematic: Consistent collection methods Timestamped: All data points timestamped Categorized: Organize by type Complete: Don't skip data points.

> scientific*method*tool/ ├── hypothesis.py # Hypothesis framework ├── experiment.py # Experiment design └── analysis.py # Analysis tools.

Science-Bitch provides a comprehensive, systematic approach to scientific research and experimentation. By implementing the complete scientific method workflow with automatic state capture, systematic data collection, and professional report generation, it enables researchers to conduct rigorous, evidence-based research with full traceability.

> Systematic Approach: Structured workflow ensures completeness Evidence-Based: All conclusions traceable to data Reproducible: State capture enables reproduction Professional: High-quality documentation Integrated: Works with WAFT ecosystem.

Hypothesis and design State snapshots (A and B) Collected data (C) Analysis results Conclusions.

Science-Bitch addresses these gaps by providing: Automatic State Capture: System snapshots at key points (A and B) Structured Data Collection: Systematic measurement recording (C) Traceable Evidence: Complete chains from hypothesis to conclusion Automated Reports: Professional PDF documentation.

A hypothesis is a testable statement that predicts the outcome of an experiment. It must be: Specific: Clear and unambiguous Testable: Can be verified or refuted Falsifiable: Can be proven wrong Measurable: Has quantifiable outcomes.

> The state capture system provides automatic snapshots of system state at key points in the experiment workflow. This enables: Before/After Comparison: Clear view of what changed Reproducibility: Can restore to initial state Change Tracking: Identifies all modifications Evidence Chain: Links changes to experiment results.

Capture Everything: Don't miss important components Use Hashes: For change detection Document Context: Include relevant information Version Control: Track state versions.

> A Complete Scientific Method Workflow Tool for Hypothesis Testing, Experimentation, and Evidence-Based Research.

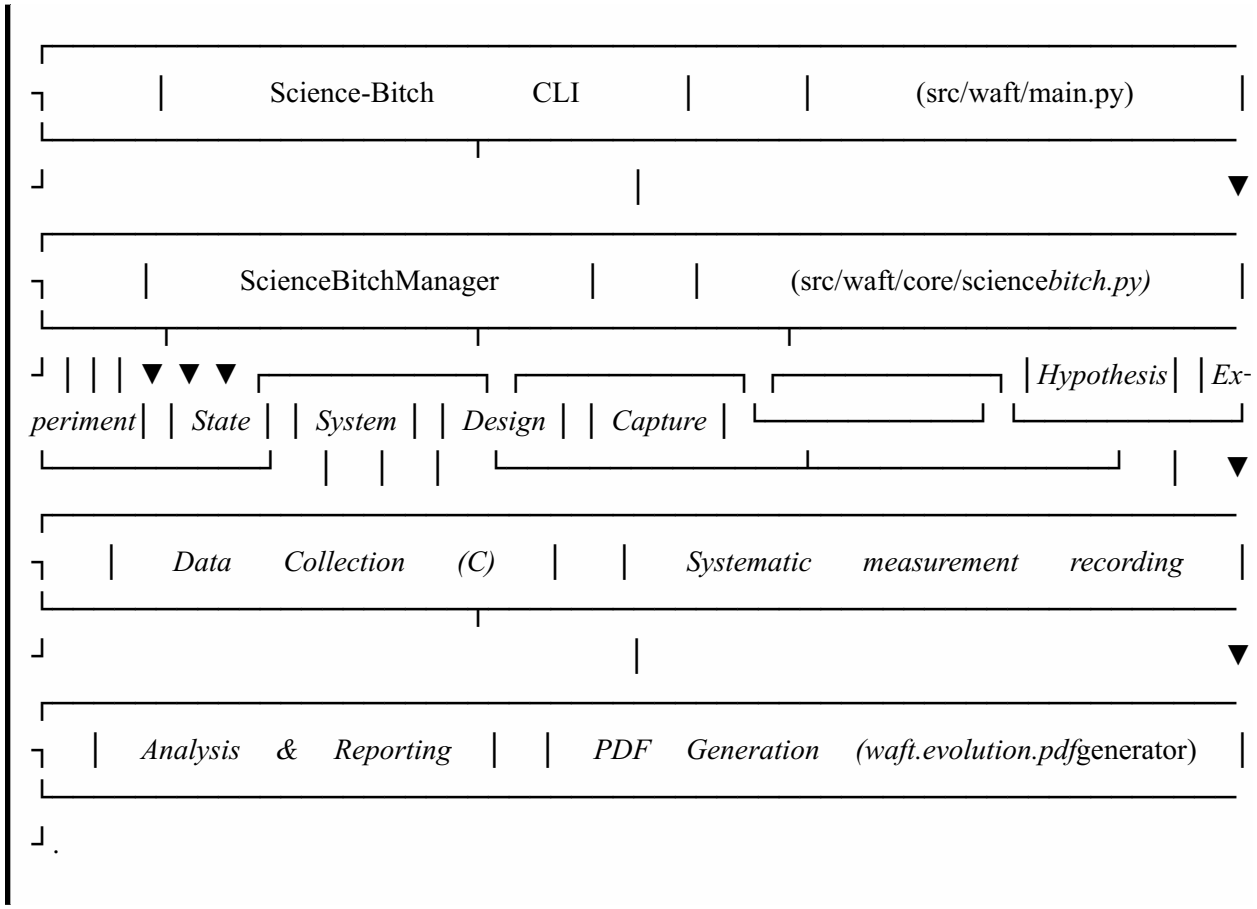
Traditional research workflows often lack: Systematic State Tracking: No clear before/after snapshots Structured Data Collection: Ad-hoc data recording Evidence Chains: Difficult to trace conclusions back to data Automated Documentation: Manual report generation.

Observe → Identify phenomenon or question Hypothesize → Form testable hypothesis Design → Create experiment with variables Capture A → Initial system state Run → Execute experiment Collect C → Data during experiment Capture B → Final system state Analyze → Verify/refute hypothesis Report → Generate conclusions Iterate → Refine and repeat.

Independent Variable: What you change or control Dependent Variable: What you measure (the outcome) Control Variables: What you keep constant.

State A: Initial system state before experiment State B: Final system state after experiment State Comparison: Identifies changes and effects.

Systematic Measurements: Recorded at specified intervals Observations: Qualitative notes and observations Metrics: Performance and behavior metrics Timestamps: All data points timestamped.



class ScienceBitchManager: def **init**(self, project*path: Path) def run*interactive(self) -> Dict[str, Any] def generate*field*guide(self) -> Optional[Path] def generate*project*status*report(self) -> Optional[Path] def* create*field*guide*content(self) -> str def* create*project*status_content(self) -> str.

Process: Identify the question or phenomenon Formulate clear, testable hypothesis Define variables (independent, dependent, control) Specify verification criteria.

Experiment Design: Run algorithm with iterations: [10, 50, 100, 200] Measure fitness score for each Capture state before (A) and after (B) Collect data during execution (C) Compare results.

> Components Captured: File system structure Configuration files Code versions Data files System metrics.

Process: Execute experiment according to design Collect data points during execution (C) Record measurements and observations Track experiment progress Handle errors and edge cases.

> Process: Capture final state snapshot Generate state hash for comparison Compare with initial state (A) Identify changes Save state to `science/experiments/[expid]/state_b.json`.

Analysis Performed: Compare states A and B Analyze collected data (C) Verify or refute hypothesis Calculate confidence level Generate conclusions Identify patterns and insights.

Additional Details

Conclusion: Hypothesis VERIFIED Confidence: 0.95 Evidence: Clear positive correlation, 100+ iterations show >10% improvement.

```
def capture_state(state_id: str) -> Dict[str, Any]: """Capture current system state.""" state = { "timestamp": datetime.now().isoformat(), "state_id": state_id, "files": capture_file_system(), "config": capture_configuration(), "code": capture_code_state(), "data": capture_data_state(), "hash": generate_state_hash() } return state.
```

The data collection framework provides systematic recording of measurements, observations, and metrics during experiment execution.

> Define Collection Points: Where to collect data Specify Intervals: How often to collect Record Measurements: Capture values Store Data: Save to structured format Timestamp Everything: All data points timestamped.

Collected data (C) is integrated with state snapshots (A and B) for comprehensive analysis: Temporal Analysis: How values change over time Correlation Analysis: Relationships between variables Statistical Analysis: Means, variances, distributions Pattern Recognition: Identifying trends and patterns.

> Define Criteria: What constitutes verification? Run Experiment: Execute according to design Collect Data: Systematic data collection Analyze Results: Compare to criteria Verify/Refute: Determine outcome Calculate Confidence: Assess certainty.

Current project state including: Goals and objectives Progress tracking Evidence collected Next steps Recommendations.

Tracks progress in `WE-260112-az3z` Links to work effort documentation Progress tracking Status updates.

Science-Bitch is accessible via: CLI: `waft science-bitch` Cursor Command: `/science-bitch` Python API: `ScienceBitchManager`.

> Workflow: Hypothesis: "New algorithm reduces execution time by 30%" Design: Compare old vs new algorithm on test dataset Capture A: Initial codebase state Run: Execute both algorithms Collect C: Record execution times Capture B: Final state Analyze: Compare performance metrics Report: Generate performance analysis.

Workflow: Hypothesis: "Parameter X=0.8 optimizes accuracy" Design: Test X values [0.5, 0.6, 0.7, 0.8, 0.9] Capture A: Initial configuration Run: Test each parameter value Collect C: Record accuracy for each Capture B: Final configuration Analyze: Identify optimal value Report: Configuration recommendations.

> Workflow: Hypothesis: "Refactoring improves maintainability without performance loss" Design: Compare before/after metrics Capture A: Pre-refactoring state Run: Execute refactored code Collect C: Performance and maintainability metrics Capture B: Post-refactoring state Analyze: Compare metrics Report: Refactoring impact analysis.

Control Variables: Keep constants constant Multiple Trials: Run multiple experiments Randomization: Reduce bias Blinding: When appropriate.

Thorough: Don't rush analysis Evidence-Based: All conclusions supported Statistical: Use appropriate methods Documented: Clear reasoning.

science/ ├── *README.md # Overview* ├── *SUMMARY.md # Summary* ├── *experiments/ # Experiment definitions* │ └── *[exp*id]/ # Individual experiment │ ├── experiment.json # Experiment definition │ ├── state*a.json # Initial state (A)* │ ├── *state*b.json # Final state (B) │ └── results.json # Experiment results ├── data/ # Collected data (C) │ └── [exp*id]/ # Data for each experiment* │ └── *data*series.json # Collected measurements ├── reports/ # Generated reports │ ├── field*guide.pdf # Usage guide* │ ├── *field*guide.md # Source markdown │ ├── project*status.pdf # Status report* │ └── *project*status.md # Source markdown └── tools/ # Helper utilities ├── generate*field*guide*pdf.py* └── *generate*status_pdf.py.

> typer: CLI framework rich: Terminal formatting weasyprint: PDF generation markdown: Markdown processing jinja2: Template rendering.

Reproducibility: Improving experiment reproducibility Scalability: Handling large-scale experiments Integration: Better ecosystem integration Usability: Improving user experience Performance: Optimizing execution speed.

> Work Effort: `WE-260112-az3zsciencebitchcommandfullscientificmethod_cli` Source Code: `src/waft/core/science_bitch.py` Documentation: `_science/README.md` Examples: `scientificmethodtool/example_usage.py` .

{ "experiment*id": "exp*20260112*001", "hypothesis": "Statement", "variables": { "independent": "...", "dependent": "...", "control": "..." }, "design": "...", "created*at": "2026-01-12T15:00:00Z" }.

PDF generation fails Install WeasyPrint: `pip install weasyprint` Check system dependencies (Cairo, Pango).

Automatic system state snapshots: File system state Configuration state Code state Data state.

> Purpose: Create experiment design based on hypothesis.

Output: Experiment design with variables, data collection plan, and state capture strategy.

> Purpose: Systematic data collection during experiment.

Output: Final state snapshot with comparison to initial state.

Current project state Goals and objectives Evidence collected Next steps Progress tracking.

Environment variables Configuration files Settings and parameters System configuration.

> Source code versions Dependencies Build artifacts Test results.

The system compares State A (initial) and State B (final) to identify:.

> Code Changes: Modified source files New dependencies Updated versions.

Performance metrics System metrics Application metrics Custom metrics.

Science-Bitch integrates with the broader WAFT ecosystem:.

System state capture Change tracking Reproducibility support.

> Command not found Ensure you're in project root Check: `waft --help` shows `science-bitch`.

Content Breakdown

Type	Count
Decisions	8
Insights	1
Actions	9
Concepts	43
Questions	7